

UNIVERSIDADE FEDERAL DO CEARÁ

CAMPUS DE ITAPAJÉ

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

LUCAS PYETRO DA SILVA NUNES

EDUARDO KEVEN BRANDÃO DA SILVA

VITÓRIA JENIFER BRAGA BARBOSA

ANTÔNIA LETÍCIA FONSECA MATOS

BOLIVAR CARNEIRO BRAGA JÚNIOR

DOCUMENTAÇÃO DO SISTEMA DE CINEMA

ITAPAJÉ

2026

LISTA DE QUADROS

Quadro 1 – Histórico de atualizações 5
Quadro 2 – Registro do rodízio de funções 6

SUMÁRIO

1 INTRODUÇÃO ..... **Erro! Indicador não definido.**
2 HISTÓRICO DE ATUALIZAÇÕES ..... **Erro! Indicador não definido.**
3 RESPONSABILIDADES DA EQUIPE..... **Erro! Indicador não definido.**
3.1 Distribuição das funções por período 6
4 ESTRUTURA DO SISTEMA ..... **Erro! Indicador não definido.**
4.1 Organização das classes..... 7
4.2 Classe Cinema ..... **Erro! Indicador não definido.**
5 DOCUMENTAÇÃO DAS CLASSES GERENCIADORAS ..... **Erro! Indicador não definido.**
5.1 Classe GerenciadorFilmes ..... **Erro! Indicador não definido.**
5.2 Classe GerenciadorSalas..... **Erro! Indicador não definido.**
5.3 Classe GerenciadorSessoes ..... **Erro! Indicador não definido.**
5.4 Classe GerenciadorClientes..... **Erro! Indicador não definido.**
5.5 Classe GerenciadorIngressos..... **Erro! Indicador não definido.**
6 PONTOS PENDENTES E VERIFICAÇÕES ..... **Erro! Indicador não definido.**
7 CONSIDERAÇÕES SOBRE A INTEGRAÇÃO ..... **Erro! Indicador não definido.**

1 INTRODUÇÃO

Este documento apresenta a organização inicial do Sistema de Cinema, desenvolvido como atividade acadêmica da disciplina de Programação Orientada a Objetos, do curso de Análise e Desenvolvimento de Sistemas da Universidade Federal do Ceará (UFC), Campus de Itapajé.

O sistema foi planejado com base na composição de classes responsáveis por representar e gerenciar filmes, salas e sessões. A documentação registra as decisões iniciais da equipe, as responsabilidades atribuídas aos integrantes e as funcionalidades previstas, de modo a facilitar a continuidade do projeto durante o rodízio quinzenal de funções.

As descrições técnicas foram elaboradas a partir das definições fornecidas pela equipe. Portanto, os métodos apresentados correspondem à estrutura planejada.

2 HISTÓRICO DE ATUALIZAÇÕES

O histórico de atualizações registra as alterações relevantes na documentação e no projeto ao longo das etapas de desenvolvimento. A cada novo período, deverão ser informados a data, o responsável e o conteúdo alterado. As seções técnicas também deverão ser revisadas para refletir as definições mais recentes.

Quadro 1 – Histórico de atualizações

Versão	Data	Responsável	Alterações
0.1	30/09/2026 a 13/10/2026	Bolivar Carneiro	Implementação das classes GerenciadorClientes e GerenciadorIngressos, responsáveis pelo gerenciamento de clientes e ingressos. Adição do método vender_ingresso() à classe Cinema, permitindo realizar a venda por meio do gerenciador de ingressos.
0.1	30/09/2026 a 13/10/2026	Antônia Letícia	Atualização do README.md e da documentação acadêmica, incluindo as novas classes, métodos e funcionalidades de venda de ingressos.
0.1	30/09/2026 a 13/10/2026	Lucas Pyetro	Planejamento da arquitetura do sistema, organização das classes gerenciadoras e definição da integração entre os componentes e o fluxo de venda de ingressos.
0.1	30/09/2026 a 13/10/2026	Vitória Jenifer	Análise do código-fonte para identificação de possíveis erros e oportunidades de melhoria na implementação do sistema.
0.1	30/09/2026 a 13/10/2026	Eduardo Keven	Identificação de inconsistências no gerenciamento de clientes e sugestão de melhorias na busca por identificador, na execução do programa e no tratamento de exceções.

Fonte: Elaborado pela equipe (2026).

Quadro 2 – Histórico de atualizações

Versão	Data	Responsável	Alterações
0.2	13/10/2026 a 13/10/2026	Bolivar Carneiro	Análise de possíveis erros no código-fonte do projeto, estudo de melhorias, além de alterações no README, para facilitar o entendimento das novas funções no sistema. Implementação do requirements.txt e do instalar_dependencias.bat.

0.2	13/10/2026 a 13/10/2026	Antônia Letícia	Vazio
0.2	13/10/2026 a 13/10/2026	Lucas Pyetro	Alterou o sistema para que ficasse mais profissional, agora o sistema possui dados persistentes que ficam salvos em arquivos .json dentro da pasta dados, que possuem subpastas: Filmes, Salas, Clientes, Ingressos, Sessoes que armazenam dados das suas respectivas classes. Toda a lógica de gerenciamento se tornou funções soltas e movidas para dentro do arquivo gerenciador.py. Adição de uma interface no terminal. A função add_ingresso verifica se a poltrona está livre, calcula o preço e bloqueia caso a poltrona esteja ocupada. O sistema impede exclusões indevidas, ou seja, um cliente que possua um ingresso não pode ser apagado. Implementação de recursos extras como o ygame e O keyboard para criar atalhos de teclado (Alt + H + P e Alt + H + S) que tocam uma música secreta (Ultra_Secret.mp3).
0.2	13/10/2026 a 13/10/2026	Vitória Jenifer	Vazio
0.2	13/10/2026 a 13/10/2026	Eduardo Keven	Alterações na documentação, exemplificando o que foi adicionado no sistema: novas classes, métodos e a integração das novas funcionalidades.

Fonte: Elaborado pela equipe (2026).

3 RESPONSABILIDADES DA EQUIPE

As funções serão distribuídas entre os integrantes e alteradas a cada duas semanas. O quadro a seguir deve registrar cada período de atuação, permitindo identificar os responsáveis sem apagar as atribuições das etapas anteriores.

3.1 Distribuição das funções por período

Quadro 1 – Registro do rodízio de funções

Período	Integrante	Função	Atividades realizadas
30/09/2026 a 13/10/2026	Antônia Letícia	Documentação	Organizou e atualizou a documentação.

30/09/2026 a 13/10/2026	Bolivar Carneiro	Programador principal	Iniciou o desenvolvimento do código-fonte, criando a estrutura inicial do sistema. Implementou as classes GerenciadorClientes e GerenciadorIngressos e adicionou o método vender_ingresso() à classe Cinema, integrando o processo de venda aos clientes e às sessões.
30/09/2026 a 13/10/2026	Lucas Pyetro	Arquitetura	Planejou a arquitetura, organizando os gerenciadores de filmes, salas, sessões, clientes e ingressos na classe Cinema; definiu a integração entre classes e o fluxo de venda, considerando disponibilidade de assentos e capacidade das salas.
30/09/2026 a 13/10/2026	Vitória Jenifer	Programadora de apoio	Atuou como programadora de apoio nas duas primeiras semanas, analisando o código-fonte para identificar possíveis erros e pontos de melhoria.
30/09/2026 a 13/10/2026	Eduardo Keven	Testes	Analisou o código sem executá-lo; identificou chamada ao método get_cliente_by_nome() não definido, ausência do bloco if __name__ == "__main__" e de entrada dos dados da venda; sugeriu tratamento de exceções.

Fonte: Elaborado pela equipe (2026).

Quadro 2 – Registro do rodízio de funções

Período	**Integrante**	**Função**	**Atividades realizadas**
13/10/2026 a 13/10/2026	Antônia Letícia	Arquitetura	Organizou e atualizou a documentação.
13/10/2026 a 13/10/2026	Bolivar Carneiro	Programador de apoio	Iniciou o desenvolvimento do código-fonte, criando a estrutura inicial do sistema. Implementou as classes GerenciadorClientes e GerenciadorIngressos e adicionou o método vender_ingresso() à classe Cinema, integrando o processo de venda aos clientes e às sessões.

13/10/2026 a 13/10/2026	Lucas Pyetro	Programador principal	Planejou a arquitetura, organizando os gerenciadores de filmes, salas, sessões, clientes e ingressos na classe Cinema; definiu a integração entre classes e o fluxo de venda, considerando disponibilidade de assentos e capacidade das salas.
13/10/2026 a 13/10/2026	Vitória Jenifer	Testes	Atuou como programadora de apoio nas duas primeiras semanas, analisando o código-fonte para identificar possíveis erros e pontos de melhoria.
13/10/2026 a 13/10/2026	Eduardo Keven	Documentação	Analisou o código sem executá-lo; identificou chamada ao método get_cliente_by_nome() não definido, ausência do bloco if __name__ == "__main__" e de entrada dos dados da venda; sugeriu tratamento de exceções.

Fonte: Elaborado pela equipe (2026).

4 ESTRUTURA DO SISTEMA

A arquitetura inicialmente definida utiliza classes auxiliares para o gerenciamento de diferentes componentes do cinema. Cada gerenciador armazena objetos de sua classe correspondente e concentra as operações previstas para esses objetos.

4.1 Organização das classes

O sistema contém as classes GerenciadorFilmes, GerenciadorSalas, GerenciadorSessoes, GerenciadorClientes e GerenciadorIngressos. Cada gerenciador mantém uma lista interna de objetos e oferece métodos específicos para consulta e gerenciamento. O GerenciadorIngressos também realiza a venda de ingressos, associando cada ingresso a um cliente e a uma sessão.

4.2 Classe Cinema

A classe Cinema reúne os cinco gerenciadores como atributos e disponibiliza métodos para acessá-los. Além disso, o método vender_ingresso(self, cliente: Cliente, sessao: Sessao, tipo: str, quantidade: int = 1, poltrona: int = None, id: int = None) encaminha a

operação de venda ao GerenciadorIngressos. O acesso aos gerenciadores permite chamadas encadeadas, conforme o exemplo a seguir:

cinema.gerenciar_filmes().adicionar_filme(...)

No exemplo, gerenciar_filmes() disponibiliza o gerenciador responsável pelo catálogo, enquanto adicionar_filme() executa a operação correspondente. As assinaturas específicas dos métodos de acesso da classe Cinema ainda não foram detalhadas no material recebido.

5 DOCUMENTAÇÃO DAS CLASSES GERENCIADORAS

As subseções apresentam as responsabilidades, os atributos e os métodos identificados no código disponibilizado pela equipe, incluindo os gerenciadores de clientes e ingressos.

5.1 Classe GerenciadorFilmes

A classe GerenciadorFilmes é responsável pelo catálogo de filmes. O atributo self.__filmes = [] armazena objetos da classe Filme, inicialmente em uma lista vazia.

Métodos implementados:

- **__init__(self)** — Inicializa a lista interna de filmes.
- **get_filme_by_id(self, id: int)** — Retorna o filme com o ID informado; se não existir, retorna False.
- **adicionar_filme(self, id: int, titulo: str, duracao: str, genero: str, sinopse: str)** — Cria e adiciona o filme se não houver outro com o mesmo ID.
- **excluir_filme(self, id: int)** — Remove o filme correspondente ao ID, quando encontrado.
- **atualizar_filme(self, id: int, n_titulo: str, n_duracao: str, n_genero: str, n_sinopse: str)** — Atualiza os atributos do filme, exceto o ID.
- **listar_filmes(self)** — Exibe ID, título, duração e gênero dos filmes cadastrados.

O identificador de cada filme deverá ser único. A atualização preservará esse identificador, conforme a decisão inicial da equipe.

5.2 Classe GerenciadorSalas

A classe GerenciadorSalas é responsável pelo cadastro e gerenciamento das salas. O atributo self.__salas = [] armazena objetos da classe Sala, inicialmente em uma lista vazia.

Métodos implementados:

- **__init__(self)** — Inicializa a lista interna de salas.
- **get_sala_by_num(self, num: int)** — Retorna a sala com o número informado; se não existir, retorna False.
- **adicionar_sala(self, num: int, cap: int, ocup: int = 0)** — Cria e adiciona a sala caso o número ainda não esteja cadastrado.
- **excluir_sala(self, num: int)** — Remove a sala correspondente ao número, quando encontrada.
- **atualizar_sala(self, num: int, cap: int, ocup: int = 0)** — Atualiza capacidade e ocupação, preservando o número.
- **listar_salas(self)** — Exibe número, capacidade e assentos ocupados.

O número identifica cada sala de forma única. O parâmetro ocup possui valor padrão zero nas

operações indicadas.

5.3 Classe GerenciadorSessoes

A classe GerenciadorSessoes é responsável pelas sessões. O atributo self.__sessoes = [] armazena objetos da classe Sessao, inicialmente em uma lista vazia.

Métodos implementados:

- **__init__(self)** — Inicializa a lista interna de sessões.
- **get_sessao_by_id(self, id: int)** — Retorna a sessão com o ID informado; se não existir, retorna False.
- **criar_sessao(self, id: int, filme: Filme, sala: Sala, data: str, horario: str, preco: float, assentos: int = 0)** — Cria e adiciona uma sessão caso o ID ainda não esteja cadastrado.
- **remover_sessao(self, id: int)** — Remove a sessão correspondente ao ID, quando encontrada.
- **atualizar_sessao(self, id: int, filme: Filme, sala: Sala, data: str, horario: str, preco: float, assentos: int = 0)** — Atualiza os dados da sessão, preservando o ID.
- **listar_sessoes(self)** — Exibe ID, filme, sala, data, horário, preço e assentos ocupados.

Cada sessão está associada a objetos Filme e Sala. O identificador da sessão deverá ser único, e o parâmetro assentos terá valor padrão zero quando não for informado.

5.4 Classe GerenciadorClientes

A classe GerenciadorClientes mantém os clientes cadastrados no atributo self.__clientes = []. Seus métodos permitem consultar, cadastrar, excluir, atualizar e listar clientes. A classe Cliente possui, entre outros, nome, idade, identificador e lista de ingressos.

Métodos implementados:

- **__init__(self)** — Inicializa a lista interna de clientes.
- **get_cliente_by_id(self, id: int)** — Retorna o cliente pelo ID; se não encontrado, retorna False.
- **adicionar_cliente(self, nome: str, idade: int, id: int = None)** — Verifica o nome e, se não cadastrado, cria e retorna o cliente.
- **excluir_cliente(self, id: int)** — Remove o cliente identificado pelo ID, se encontrado.
- **atualizar_cliente(self, id: int, novo_nome: str = None, nova_idade: int = None, novo_id: int = None)** — Atualiza os dados informados e retorna o cliente.
- **listar_clientes(self)** — Exibe nome, idade e quantidade de ingressos de cada cliente.

Observação técnica: no código recebido, adicionar_cliente() chama get_cliente_by_nome(), mas esse método não está definido na classe apresentada. A equipe deve verificar essa dependência.

5.5 Classe GerenciadorIngressos

A classe GerenciadorIngressos armazena os ingressos em self.__ingressos = []. Além das operações de consulta, inclusão, remoção e listagem, ela executa a venda, verificando cliente, sessão, quantidade, disponibilidade e poltronas, e atualizando a ocupação da sessão.

Métodos implementados:

- **__init__(self)** — Inicializa a lista interna de ingressos.
- **get_ingresso_by_id(self, id: int)** — Consulta o ingresso pelo ID; se não encontrado, retorna False.
- **adicionar_ingresso(self, id: int, preco: float, poltrona: int, tipo: str, quant: int = 1, cliente: Cliente = None, sessao: Sessao = None)** — Cria o ingresso com ID não repetido e o associa à lista do cliente, quando informado.
- **remover_ingresso(self, id: int)** — Remove o ingresso, desvincula-o do cliente e ajusta a ocupação da sessão, quando aplicável.
- **listar_ingressos(self)** — Exibe ID, cliente, tipo, poltrona, quantidade e preço.
- **vender_ingresso(self, cliente: Cliente, sessao: Sessao, tipo: str, quantidade: int = 1, poltrona: int = None, id: int = None)** — Valida a venda, atribui IDs e poltronas, cria ingressos individuais, atualiza assentos ocupados e retorna os ingressos criados.

A venda gera um ingresso individual para cada poltrona solicitada. O método da classe Cinema com a mesma assinatura delega a operação ao GerenciadorIngressos.

6 PONTOS PENDENTES E VERIFICAÇÕES

As classes Cliente e Ingresso e seus respectivos gerenciadores já estão presentes no código. Permanecem pendentes a correção da chamada ao método get_cliente_by_nome() na classe GerenciadorClientes e a implementação do método exibir_menu() da classe Cinema, que ainda contém apenas pass. Também é necessário verificar o funcionamento do sistema por meio de testes.

7 CONSIDERAÇÕES SOBRE A INTEGRAÇÃO

A classe Cinema instancia os gerenciadores de filmes, salas, sessões, clientes e ingressos. Os métodos gerenciar_filmes(), gerenciar_salas(), gerenciar_sessoes(), gerenciar_clientes() e gerenciar_ingressos() retornam os respectivos gerenciadores. O método vender_ingresso() da classe Cinema delega a venda ao GerenciadorIngressos, que vincula ingressos a clientes e sessões.

À medida que o desenvolvimento avançar, deverão ser registrados os contratos de integração acordados entre os integrantes, as alterações nas assinaturas dos métodos e as novas dependências entre as classes. A documentação deverá refletir o estado efetivo do projeto, distinguindo decisões previstas de funcionalidades implementadas.